

Refining Network-Based Framework for Accurate Actin Filament Identification

Senyu Zhu

M. Sc. Bioinformatics

Institute of Biochemistry and Biology, University of Potsdam

Supervisors:

Dr. Jacqueline Nowak

Prof. Dr. Zoran Nikoloski

Date of submission:

October 14, 2024

Abstract

The Graph of Filaments over Time (GraFT) framework, developed by Østerlund et al., is a network-based method for tracing and tracking filamentous structures in cytoskeleton networks. Actin filaments are crucial in many parts of the cell, including cell motility, structural integrity, and intracellular transport. They form complex and dynamic networks that are challenging to trace, particularly in regions of filament overlap and closed structures. Despite its success in tracing actin filaments, GraFT struggles with these more complex structures. In this project, we extend the GraFT algorithm to improve its robustness by incorporating additional constraints—specifically, filament intensity and thickness—into the tracing process. These constraints complement the original angle-based constraint, allowing the algorithm to more accurately distinguish overlapping filaments. In addition to the constraints, optimizations have been made in the code to allow tracing of closed shapes. The performance of the enhanced algorithm was evaluated using validation sets that were manually traced and labelled to provide ground truth data for quantification. The updated GraFT was applied to both synthetic image data and real imaging data from *Arabidopsis thaliana* seedlings, imaged at three developmental stages—young, expanding, and mature cells—using confocal microscopy. Results show that the extended GraFT algorithm outperforms the original version on synthetic images, given the optimal parameter values. This improved robustness makes the algorithm a more reliable tool for studying actin filaments and other cytoskeleton components, with potential applications across a range of biological contexts involving filamentous networks.

1 Introduction

The cytoskeleton is a fundamental component of eukaryotic cells, providing structural support, facilitating intracellular transport, and enabling cellular movements [1]. It is composed of three main types of filamentous proteins: microtubules, intermediate filaments, and actin filaments. These structures vary in their function, diameter, and persistence length [2].

First, microtubules are the thickest filaments, measuring approximately 25 nm in diameter. With a persistence length of 0.2–9 μm , they are the most rigid of the three filament types and form nearly straight structures [3,4]. Second, intermediate filaments have a diameter of about 10 nm [5]. While they play a crucial role in animal and yeast cells by providing mechanical strength against shear stress, they are absent in plant cells [2,6]. Lastly, the most versatile and dynamic elements are the actin filaments (F-actin), which are the thinnest, measuring 6–9 nm in diameter [7]. The persistence length of actin filaments, approximately 10–17 μm , reflects their flexibility, allowing them to bend and form various architectures within the cell [1,4]. Their ability to rapidly polymerize and depolymerize enables cells to change shape, move, and divide [8]. The dynamic nature of actin is governed by a balance of assembly and disassembly processes, regulated by a host of actin-binding proteins that modulate filament nucleation [9], elongation [10], branching [9], and crosslinking [7,11].

Given the intricate architecture of the cytoskeleton, it can be represented as a network [6], which is widely used across various scientific fields to extract meaningful information from complex and heterogeneous data sets [12]. In this context, network-based analysis of imaging data offers a novel perspective to quantify and study traits and functions in different biological systems [13]. Breuer et al. have developed an image-based, network-driven framework that effectively quantifies the dynamic properties and functions of the actin cytoskeleton [6,14].

Traditional quantitative characterization of these networks is done via human annotation [15], but manual tracing and labeling are labor-intensive and prone to observer bias [16, 17]. Consequently, computational tools have become indispensable for analyzing cytoskeletal structures [17, 18]. Current tools for tracing filamentous structures can be categorized into two main types: those that trace filaments directly on image data and those that work on a network created from image data [19]. Tools such as SOAX [20] and TSOAX [21], which utilize Stretching Open Active Contours (SOACs) [22], and FilamentSensor 2.0 [23], trace directly on image data. In contrast, common network-based tools include FiNTA [24], NEFI [25], CytsoSeg 2.0 [14, 26], and DeFiNe [27]. Additionally, there is an open-source image processing package in Fiji [28] called FilamentDetector that can also trace filamentous structures [29].

Recent advancements in these frameworks have successfully quantified microtubule organization and dynamics [30]; however, challenges remain in accurately tracing actin filaments, mainly due to their complex structures and rapid dynamics [26]. The flexibility of actin filaments allows them to form highly dynamic, overlapping networks, including curved or looped configurations [19, 31]. These structures pose significant challenges for tracing and analyzing actin networks, as tracing accuracy decreases with increased network density [32].

The Graph of Filaments over Time (GraFT) framework, developed by Østerlund et al., is the primary focus of this project [19]. GraFT employs a network-based representation of images, converting filamentous structures into graph models for computational analysis. Its key component for tracing filaments involves performing a Depth-First Search (DFS) traversal [33], while adhering to certain constraints or conditions. By using a constrained Depth-First Search (cDFS) combined with a minimal path cover approach, GraFT effectively traces and tracks filaments over time. The primary constraint in this method is the angle between filament segments, which helps maintain filament continuity during tracing. GraFT has outperformed earlier tools like TSOAX [21], FilamentSensor 2.0 [23], and DeFiNe [27] in synthetic filamentous networks [19].

However, despite its innovative approach, GraFT faces limitations when dealing with more complex structures. Specifically, it struggles with closed shapes such as circles or loops, often resulting in the over-segmentation of filaments. Additionally, relying solely on the angle-based constraint makes it difficult for the algorithm to distinguish between overlapping filaments. When filaments overlap and form an angle between 140° and 170° , they satisfy the angle constraint, which can lead the algorithm to incorrectly interpret overlapping filaments as a continuous path, resulting in under-segmentation.

This project aims to address these challenges by enhancing GraFT's robustness through: (i) introducing additional constraints into the tracing process, (ii) implementing a scoring system based on these constraints to decide which edge segments to trace, (iii) modifying the node placement function and the cDFS function to trace closed shapes, (iv) creating synthetic images and labeling them to establish gold standards, (v) generating gold standards from real images, and (vi) comparing the predicted filaments to the ground truth images to evaluate the performance of the extended algorithm. Results show that the extended algorithm successfully traces closed shapes and, to some extent, complex structures that the original GraFT algorithm could not handle.

2 Materials and methods

2.1 Data

The biological image used for this project was provided by Østerlund [34]. We selected one image from a dataset that includes a time series of *Arabidopsis thaliana* seedlings. To visualize the actin cytoskeleton, it was tagged with the mNeonGreen-FABD2 marker. The images capture seedlings at three different developmental stages, based on their position along the stem: the upper part, which contains young cells; the middle part, characterized by expanding cells; and the bottom part, where mature cells are located near the root. The images were acquired using an inverted 3i Marianas Spinning Disk confocal microscope equipped with a 60x objective.

2.2 Benchmark in silico data and validation sets

In addition to the biological dataset, we created a synthetic image that was manually drawn on an Android tablet. This synthetic image serves as the main benchmark for the GraFT framework. Validation sets were created by manually tracing the biological dataset, and these are used as ground truth images to compare the performance of the labeled images generated by the GraFT framework. Each ground truth image consists of multiple layers, ensuring that filaments do not overlap. This guarantees that individual filaments are labeled accurately in the evaluation process.

2.3 Software

The software used for tracing the ground truth sets and creating the synthetic image on the tablet includes two free applications: ibisPaint X and Samsung Notes.

The source code for the original GraFT algorithm is available on Østerlund's GitHub page (<https://github.com/Oesterlund/GraFT>). The framework is implemented in Python 3 [35], utilizing several key modules, including numpy, pandas, matplotlib, skimage, scipy, and simplification.

2.4 Evaluation metrics

We created a pipeline that evaluates how well the predicted image labelled the filaments. This is done in several steps:

1. To evaluate how well the predicted images label filaments, we first processed the ground truth images, which were manually traced and labeled on a tablet. Each ground truth image contained multiple channels, including an alpha channel, which needed to be removed for proper processing. We converted these 4-channel images into grayscale, representing pixel intensity values ranging from 0 to 255. The ground truth images were created on a white background. To ensure that the background pixels were properly represented with a value of zero (crucial for distinguishing filaments from the background), we inverted the grayscale images. After inversion, any background noise was removed using a thresholding technique, filtering out pixels below a certain intensity value. This step eliminated minor artifacts and ensured that only meaningful filamentous structures were retained and labeled for further analysis.
2. After processing the individual ground truth layers, the next step was to create a labeled ground truth image by merging the manually labeled layers. Each layer represents distinct filamentous structures in the ground truth data. While merging the layers, overlaps between filaments can occur. These overlaps

introduce additional labels, as the values of the two labels are added together. As a result, different filaments may be incorrectly over-labeled.

To avoid over-labeling, we assigned unique labels to each filament across layers before merging them and saved these unique labels in a separate array. This ensured that only the correct filament labels were retained. We identified all filaments in each layer using connected-component labeling and offset the labels so that each filament had a unique identifier.

The final output of this process is a merged ground truth image where each filament is assigned a unique label.

3. Once the ground truth image had been processed and labeled, we moved on to extracting the pixel coordinates of each filament from the predicted graph network. The predicted graph represents filaments as edges between nodes in a graph structure, with each edge associated with a filament tag.

We began by creating a mapping between each node in the graph and its corresponding pixel coordinates in the image. Using this mapping, we extracted the coordinates of the pixels that make up each filament. These pixel coordinates were stored in a dictionary, mapping each filament tag to the set of pixel coordinates that form the filament.

Next, we used the extracted pixel coordinates to create the labeled predicted image. This image is a 2D array with the same dimensions as the original image, where each pixel is labeled according to the filament it belongs to. Filament tags were assigned to all pixels that correspond to filaments in the predicted graph, resulting in a labeled predicted image that could be compared to the labeled ground truth image for evaluation.

4. To quantitatively assess the accuracy of the filament tracing algorithm, we computed the overlap between the predicted and ground truth filaments by constructing a confusion matrix. The confusion matrix provides a pixel-wise comparison, counting the number of overlapping pixels between each predicted filament and its corresponding ground truth filament. This matrix is crucial for determining how well the algorithm identifies and traces individual filaments.

Each entry in the matrix represents the number of pixels where a specific predicted filament overlaps with a particular ground truth filament. The matrix has dimensions corresponding to the number of predicted filaments and the number of ground truth filaments, with the value at position $[i, j]$ representing the number of overlapping pixels between predicted filament i and ground truth filament j .

Once the confusion matrix was constructed, we matched the predicted filaments to the ground truth filaments based on maximum overlap. This step ensures that each predicted filament is associated with the ground truth filament it most closely resembles. We used the Hungarian algorithm (also known as the linear sum assignment algorithm) to solve this matching problem optimally.

The algorithm finds the best match between predicted and ground truth filaments by maximizing the pixel-wise overlap. Since the Hungarian algorithm minimizes cost, we negated the values in the confusion matrix to transform the problem into one of maximization. The linear sum assignment formula is given by Eq 1:

$$\min \sum_{i=1}^n \sum_{j=1}^m -C_{ij}x_{ij} \quad (1)$$

where C_{ij} represents the values from the confusion matrix, and x_{ij} is 1 if predicted filament i is assigned to ground truth filament j , otherwise 0.

The result is a set of matched filaments, where each predicted filament is assigned to a ground truth filament based on the greatest number of overlapping pixels.

5. The matches from the previous step were used to count true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN). We created two functions that calculate the evaluation metrics differently. The first function calculates the F1-score (Dice coefficient) ² and the Intersection over Union (IoU, or Jaccard Index) ³ by counting TPs, FPs, and FNs based on the number of pixel overlaps. This provides a quantitative measurement of performance.

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2)$$

where $\text{Precision} = \frac{TP}{TP+FP}$ and $\text{Recall} = \frac{TP}{TP+FN}$.

$$IoU = \frac{TP}{TP + FP + FN} \quad (3)$$

The second function calculates the F1-score and the Matthews correlation coefficient (MCC) ⁴ by counting TPs, TNs, FPs, and FNs directly from the entries in the confusion matrix. TPs are incremented when a matching entry is found in both the row and column of the matrix. FPs arise from over-segmentation, where the predicted filaments assign multiple labels to a single ground truth label, resulting in multiple entries in a single column of the confusion matrix. All entries in that column, except for one (the TP), are considered FPs.

$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (4)$$

Conversely, FNs result from under-segmentation, where the predicted image assigns one label to multiple ground truth labels, leading to multiple entries in a single row. All entries in that row, except for the one TP, are considered FNs. TNs are all the remaining entries that were not classified as TP, FP, or FN.

2.5 Key components of GraFT

The GraFT framework, developed by Østerlund et al., consists of five key steps for tracing filamentous structures (FSs) from single or time-series images [19]:

1. Image Pre-processing: The input images are pre-processed to enhance filamentous structures and reduce noise. This involves contrast enhancement, median filtering, and the application of the Frangi vesselness filter, which highlights linear structures and suppresses circular noise.
2. Segmentation and Skeletonization: After pre-processing, the images are binarized using hysteresis thresholding, with the threshold determined by Otsu's method. The images are then skeletonized using the Zhang-Suen algorithm, reducing all structures to a one-pixel width. Small objects are removed to prepare the image for network representation.

3. Network Representation: The skeletonized image is converted into a network by representing filamentous structures as nodes and edges. Initially, nodes are placed at pixel junctions and filament ends, where the pixel neighborhood contains either one or three or more neighboring pixels. For simplicity, all nodes not of degree two are considered "dangling nodes." The remaining degree-two nodes represent edges. To improve network accuracy for filaments with low persistence length, nodes are placed along all pixel edges. These nodes are then downsampled by merging closely spaced nodes using the Visvalingam-Whyatt (VW) algorithm. The VW threshold determines the degree of node merging and is adjusted to accurately represent the bending of filament structures.
4. Filament Identification (Tracing): Individual filaments are traced by performing a constrained Depth-First Search (cDFS) on each connected component of the graph. Filament paths are determined by maximizing length and minimizing sharp bends, controlled by an angular threshold. This process identifies the longest possible paths, which are iteratively refined to cover all filaments in the network.
The minimum angle threshold is calculated by applying lower and upper thresholds on each connected component of the network. All angles within the component are considered, and the smallest angle greater than 90° is taken as the minimum threshold. Additionally, an upper threshold of 140° is applied. If all nodes have an angle greater than 140° , the 140° angle is used as the minimum threshold.
5. Filament Tracking: This step, which tracks filamentous structures over time, is beyond the scope of this project and was not implemented.

2.6 Extension of GraFT

The extension of GraFT was primarily applied to the third step (network representation) and the fourth step (tracing of filamentous structures).

2.6.1 Node placement

After the image is processed in the first two steps (pre-processing and segmentation), nodes are placed on the enhanced image using the `project_edges` function. First, all the dangling nodes are placed using a 3×3 kernel that extracts the nearest 8-pixel neighbors of the center node. If the center pixel has one or three or more neighboring pixels, a "dangling" node is placed at the center.

The remaining nodes, which enhance the flexibility of filament structures, are placed using the Visvalingam-Whyatt (VW) algorithm. However, we encountered that this algorithm only performs node placement for structures containing dangling nodes. Consequently, closed networks (i.e., connected components) such as circles or triangles are not processed by the node placement function, as they only contain nodes of degree two. Thus, those closed shapes lack dangling nodes.

In the original function, several copies of the image are made, one of which is called `filtU`. This copy is a matrix where the kernel iterates to place nodes. Once a node is placed, the corresponding pixel is excluded by setting its value to 0. After the initial run of the original `project_edges` function, the pixels of the unprocessed closed shapes remain in the `filtU` matrix.

To address this, we extended the function to iterate through all the remaining connected components in `filtU`. Starting from a random node in each connected component, we perform a Depth-First Search (DFS) to traverse the closed shape. All the pixel coordinates are saved in an array and down-sampled using the VW algorithm.

Once this process is complete, all nodes are processed, and the `filtU` matrix is blank with only 0s as values.

2.6.2 Thickness calculation

For the constrained DFS, we planned to add additional constraints, one of which is the thickness constraint. The thickness of the filaments was not considered in the original code. Therefore, we first extended the `make_graph_mask` function, which constructs a graph from an annotated image, an intensity image, a mask of edges, and a DataFrame of node positions.

To compute the thickness of the filaments, we used the "Euclidean Distance Transform" (EDT) function. This function calculates, for each foreground pixel (with a value of one or higher), the minimum number of foreground pixels required to reach the nearest background pixel (value 0). The result is a distance map where each pixel's value represents its distance from the nearest background pixel. To account for both sides of the filament, representing its full thickness, we multiplied all the values by 2. We then skeletonized the filaments to isolate the center pixels, which retain the thickness measurements. Finally, we calculated the minimum, maximum, and average width for each edge segment, which are delimited by the nodes.

The resulting graph represents the network of filaments with various edge attributes, the most important of which are:

- The rod-like distance between two nodes, known as `edist`, calculated using Euclidean distance.
- The actual distance between two nodes, known as `fdist`, determined by counting pixels.
- Capacity, known as `capa`, representing the intensity.
- The newly added thickness attribute, known as `width`.

2.6.3 Constrained Depth-First Search

Overview with pseudocode: The constraint-based Depth-First Search (cDFS) is used to trace and traverse the network of actin filaments. The following pseudocode presents the logic behind the proposed algorithm. The lines spanning from 13-17, 27-28 and 33-34 were added as part of the extension.

Algorithm 1: Constrained DFS for Filament Extraction

Data : graph_s – graph from image data
lgG.V – line graph from graph
ImgBl – image with the added nodes from the VW algorithm
pos – list of node positions
angle – minimum angle allowed
overlap_allowed – minimum overlap allowed

Result : graphTagF – graph with tagged filament values

```
1 graphTag ← copy of graphs;
2 graphTag ← mark dangling edges in graphTag;
3 connected_components ← list of connected components from graphs;
4 tag ← 0;
5 to_add ← empty list;
6 while connected_components is not empty do
7   c ← remove first component from connected_components;
8   G ← subgraph of graphs for component c;
9   angle_min_val ← minimum angle using min_angle(G, ImgBl, pos, angle);
10  al_pd ← angles between edges from line graph lgG.V;
11  Initialize intensity, thickness tolerances based on data from G;
12  Initialize dangling_nodeS from nodes with degrees 1, 3, and 4;
13  if dangling_nodeS is empty then
14    Connected component is a closed shape;
15    Consider every two nodes of degree 2 as dangling_nodeS;
16  if there are two dangling nodes and they are neighbors then
17    Consider every two nodes of degree 2 as dangling_nodeS;
18  while dangling_nodeS is not empty do
19    source ← remove first node from dangling_nodeS;
20    stack ← initialize DFS stack from source;
21    visited ← empty set;
22    while stack is not empty do
23      parent, depth_now, children ← top of stack;
24      for each child of parent do
25        if child not in visited then
26          Calculate intensity, thickness, and angle between parent and child;
27          if all conditions meet thresholds then
28            update path_keep and visited;
29          if depth allows then
30            add child to stack;
31          if child is a dangling node then
32            save current path;
33      if connected component is a closed shape and we reached the starting
        node then
34        Execute lines 26–32;
35      if child is already visited but part of a complex structure then
36        continue traversal;
37  Extend path_full with all remaining individual edges from G;
38  Compute best paths based on length and select longest paths;
39  for each pair of paths in path_keep do
40    check for overlap;
41    Remove overlapping edges exceeding overlap_allowed;
42  Tag edges and remove non-overlapping ones;
43 graphTagF ← create multigraph with tagged edges;
44 return graphTagF;
```

272

Closed shapes: With the updated `project_edges` function that places additional nodes on both open and closed shapes, we can use the cDFS algorithm to trace the networks. However, initial testing with the original cDFS showed inconsistent results. We discovered that the original cDFS function only starts tracing from a dangling node (i.e., a node of degree 1 or 3 and higher). To address this, we extended the function to add every second node of degree 2 to the dangling list if the shape is closed. This ensures performance efficiency, as adding every node could lead to performance issues due to the DFS iterating through all dangling nodes.

In some cases, the shapes have two dangling nodes that are neighbors, which function as a single dangling node. This presents the risk that not all paths are considered by the cDFS algorithm. By adding every second node of degree 2 to the dangling list, we ensure that the entire connected component is traversed.

Additionally, the original code only considers each node once. However, in closed shapes, the starting node must be considered at least twice to close the tracing path properly. Otherwise, the algorithm leaves the path open. To resolve this, we added a conditional block that specifically reconsiders the starting node if the connected component is a closed shape and if it satisfies the constraints.

Constraints: The original cDFS function includes an angle constraint. To make the algorithm more robust against complex network structures, we introduced two additional constraints: intensity and thickness. These attributes are retrieved from the graph network, and we calculate thresholds for both intensity and thickness. To determine the appropriate thresholds, we experimented with various methods, including arbitrary values, the mean and standard deviation, the median and median absolute deviation (MAD), the median and interquartile range (IQR), percentiles, and a combination of the median and percentiles.

Once the tolerance values for intensity and thickness are obtained, the algorithm traverses the network, calculating the difference in intensity and thickness between two edge segments at each step. If the differences fall within the respective tolerance values, the segments are more likely to be included in the path.

Grading system: We implemented a scoring system that enables granular decision-making when considering edge segments for inclusion in paths. This system consists of several components: the intensity coefficient, the thickness coefficient, the angle coefficient, the total score, and the score threshold. Whenever an edge segment satisfies one of the constraints, the corresponding coefficient is added to the total score. Each constraint has three possible outcomes. If the edge segment satisfies the constraint by a large margin, additional points are added to the total score. If it does not satisfy the constraint, no points are added. Finally, if the constraint is not satisfied by a large margin, a certain value is subtracted from the total score.

Once the total score is greater than or equal to the score threshold, the edge segment will be included in the path.

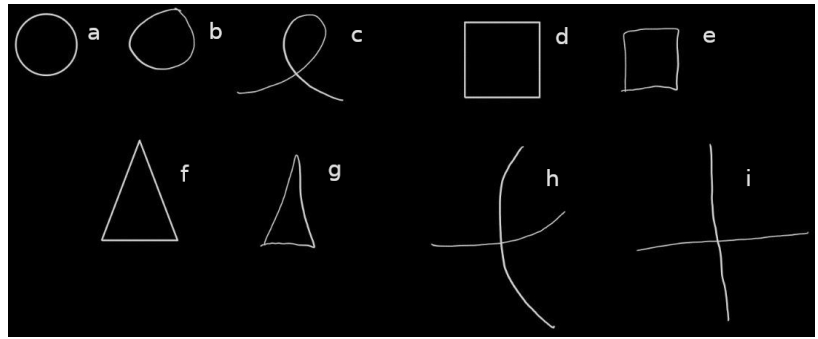
3 Results

3.1 In silico data 1

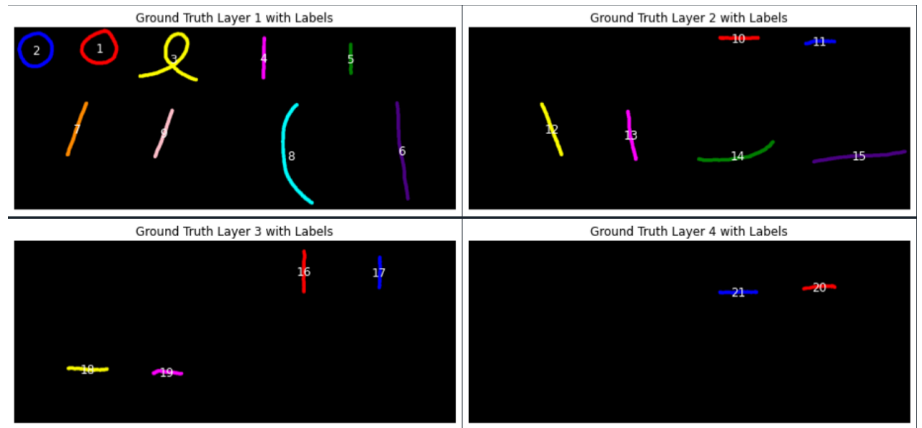
The first synthetic image was drawn using the Samsung Notes application. The image was converted to grayscale, and the intensity was inverted so that the background values are 0. The image contains multiple shapes: a, b, d, e, f, and g are closed shapes, while c, h, and i are open shapes (Fig. 1a). The parameter `eps`, which controls how much the VW algorithm simplifies node placement, was set to 200. The coefficients added to the total score when considering two edge segments were 3 for both the intensity and thickness constraints, and 5 for the angle constraint. The score threshold was set to 8. Additionally, the angle coefficient was added again to the total score if the angle between two edge segments exceeded 165° , indicating an almost straight line between them.

Shapes with major bends, including a, b, and c, exhibited significant differences in intensity and thickness between their edge segments. This variability was due to the type of digital pen used in the Samsung Notes application. Methods such as the percentile or MAD (Median Absolute Deviation) to calculate intensity and thickness tolerances proved challenging. Instead, we used arbitrary values for these tolerances that yielded optimal results when considering all three constraints. The intensity tolerance was set to 0.17, and the thickness tolerance was set to 0.27.

After tracing the in-silico image in multiple layers to avoid overlap, the layers were pre-processed and labeled (Fig. 1b).



(a) Manually drawn test data, including open and closed shapes.

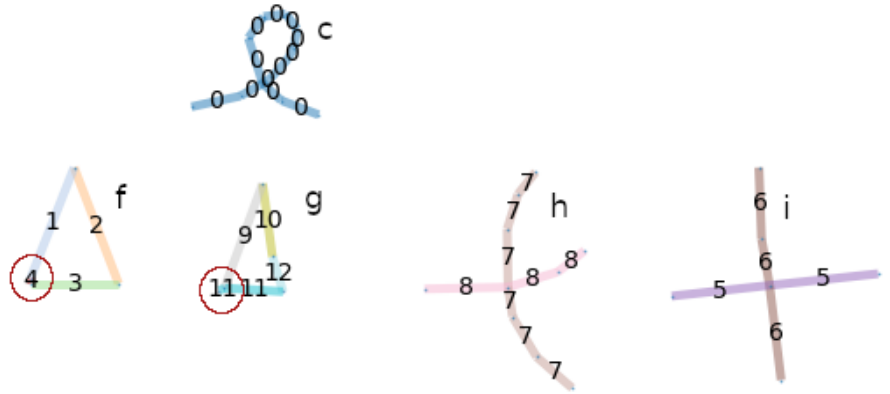


(b) Ground truth image layers.

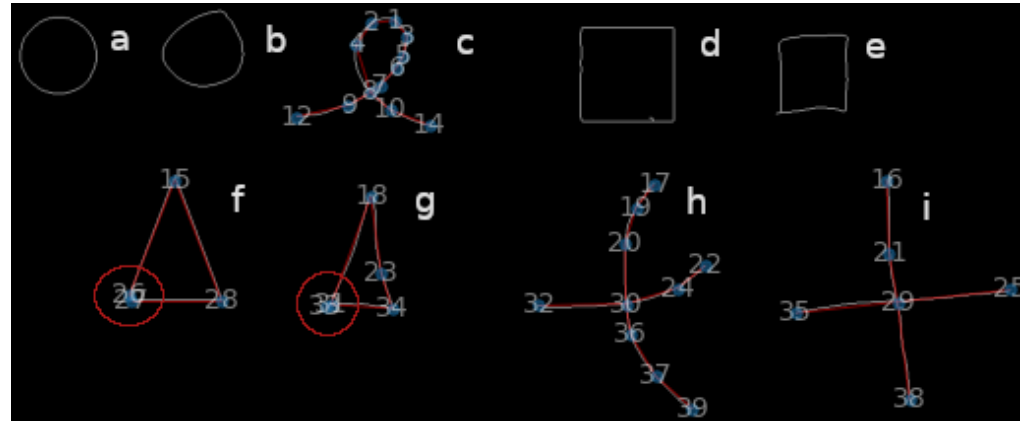
Fig 1. Synthetic image 1.

3.1.1 Performance

Original GraFT: Using the original GraFT algorithm on the in-silico data, only 5 of the 9 shapes were traced. Figure 2 shows that shapes a, b, d, and e were not included. The triangles (f, g) are also considered closed shapes. They usually only have pixels of degree 2. However, the tracing from the Samsung Notes application introduced small artifacts and noise. Consequently, the framework placed a node on the bottom left corner of one of the triangles, giving it a degree of 1, which is considered dangling.



(a) Predicted image from in-silico data 1 with the filament labels.



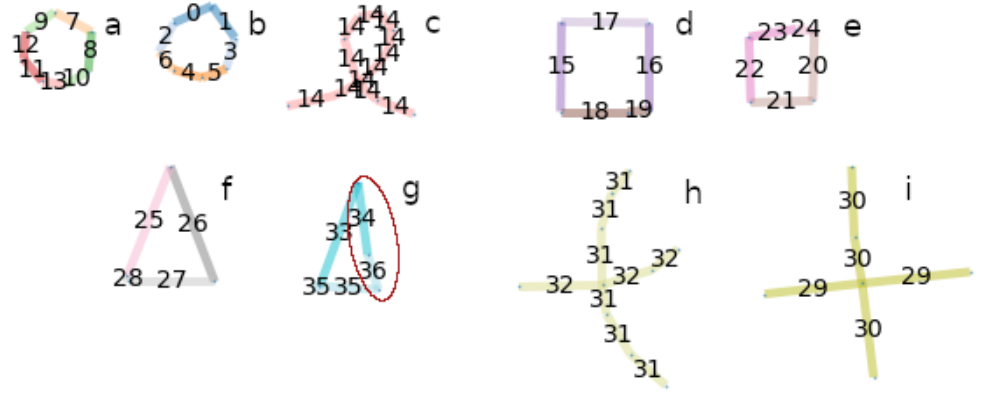
(b) Predicted image from in-silico data 1 with the dangling and extended nodes.

Fig 2. Original GraFT performance on in-silico data 1

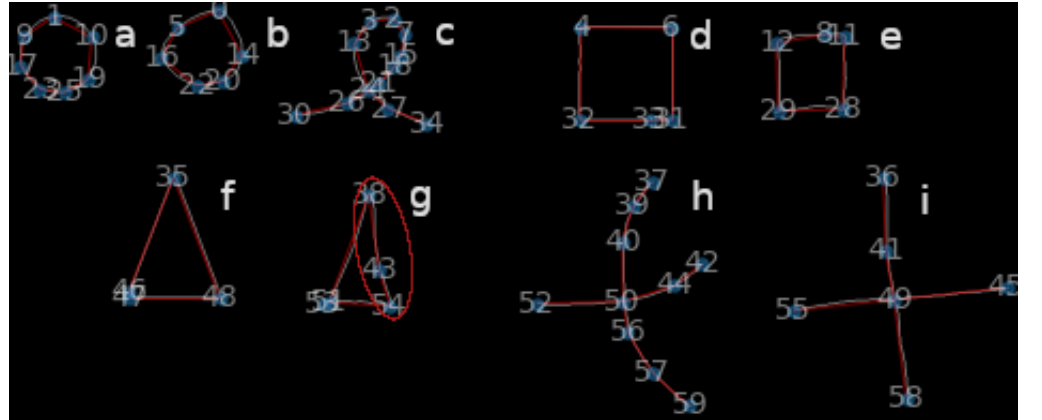
Closed shape extension: After extending the VW algorithm to include closed shapes, shapes a, b, d, e, f, and g became over-segmented (Fig.3a). First, the cDFS algorithm only starts tracing from a dangling node and only accepts a path if it reaches another dangling node. Since the closed shapes (a, b, d, and e) have no dangling nodes, the different edge segments forming a path cannot be treated as a continuous path. As a result, each edge segment is assigned its own filament tag.

Additionally, triangle g normally satisfies the angle constraint between edge segments 34 and 36. However, the only dangling nodes, 51 and 53, are clustered at the bottom left of the triangle. The cDFS algorithm starts at one of these dangling nodes, such as node 51. If it passes the angle constraint, it proceeds to node 53. From there, it can either move to node 38 or node 54. In both cases, the algorithm cannot proceed to node 43 because the angle is less than 90° (Fig.3b). As a result, the algorithm never reaches node 43, leading to over-segmentation.

To address this issue, we treated every second node of degree 2 as a dangling node, which increases the range of starting points for the cDFS algorithm.



(a) Predicted image from in-silico data 1 extended with closed shapes.



(b) Predicted image from in-silico data 1 extended with nodes in closed shapes.

Fig 3. Original GraFT performance on in-silico data 1 extended to include closed shapes.

Angle constraint: Once additional nodes were considered as dangling nodes and the starting node was considered twice, the cDFS algorithm, using only the angle constraint, was able to trace closed shapes. However, shape e in Fig. 4 remains under-segmented, as each side should be represented by a separate label. Instead, only the left filament (label 9) is correctly labeled. The minimum angle threshold is close to 90° , and the edge segments labeled with tag 8 all form angles greater than this threshold. As a result, all of these edge segments are considered as a single path.

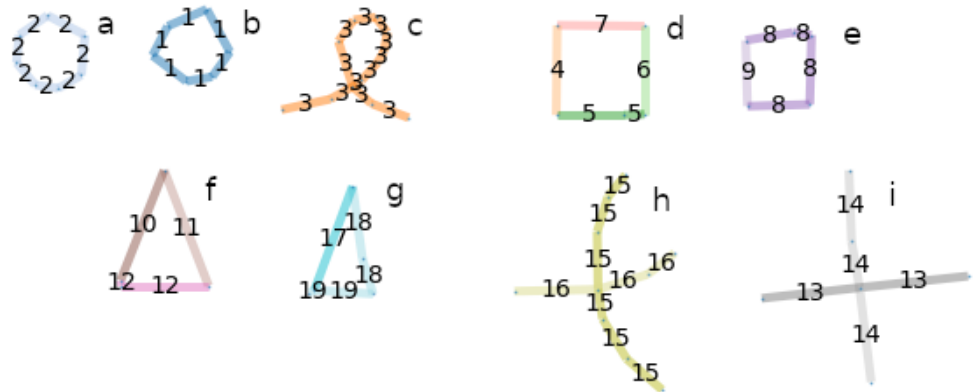


Fig 4. Predicted image from in-silico data 1 extended with closed shapes, using the angle constraint.

Angle and intensity constraints: In addition to the angle constraint, we added an intensity constraint to the tracing process. The inclusion of the intensity constraint improved the tracing of shape e, correctly labeling three of its four filaments. However, shapes a, b, f, g, and h showed worse performance, with all of them being over-segmented (Fig. 5).

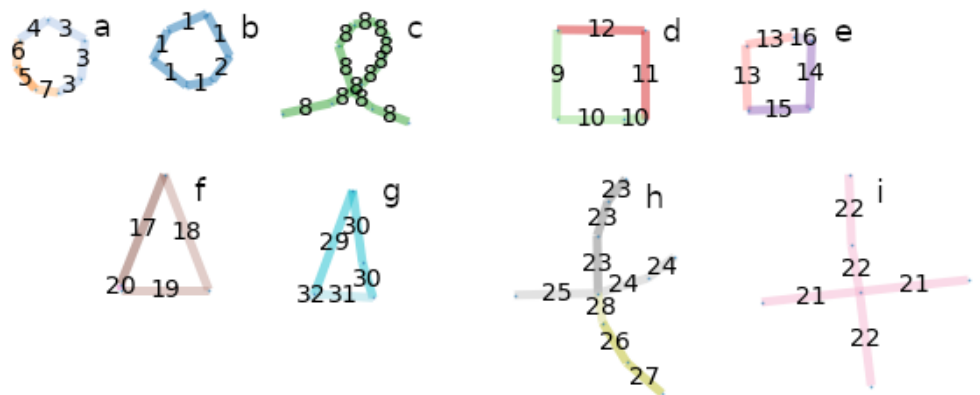
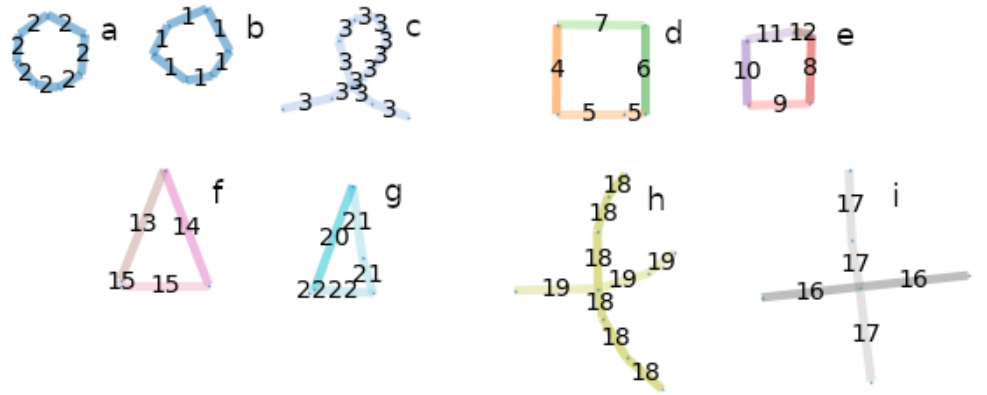


Fig 5. Predicted image from in-silico data 1 extended with closed shapes, using the angle and intensity constraints.

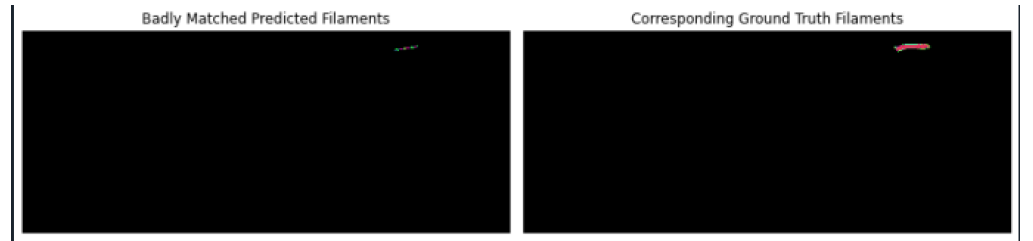
Angle and thickness constraints vs all three constraints: Using the angle and intensity constraints (Fig. 6), or all three constraints—angle, thickness, and intensity (Fig. 7)—demonstrated the highest performance. Both approaches successfully labeled all the shapes correctly, except for shape e.

The cDFS algorithm with the angle and intensity constraints introduced an additional label on ground truth filament 11, which was counted as a false positive, leading to over-segmentation (Fig. 6c). In contrast, the cDFS incorporating all three constraints correctly labeled shape e in terms of the absolute number of labels, resulting in no false positives or false negatives (Fig. 7c). However, the predicted labels 8 and 11 were identified as poor matches due to excessive overlap in label 8 and insufficient overlap in label 11 (Fig. 7b).

To quantify the performance of these two approaches, we calculated evaluation metrics based on pixel overlaps and the number of correctly labeled filaments. The first metric provides a quantitative assessment of performance, while the second offers a qualitative evaluation by measuring how accurately the filaments were labeled.



(a) Predicted image from in-silico data 1 extended with closed shapes, using the angle and thickness constraints.

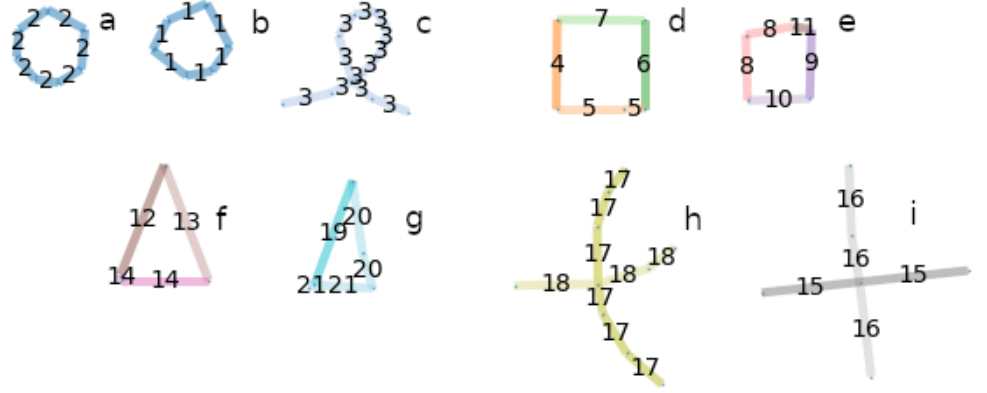


(b) Plot highlighting the badly matched filaments.

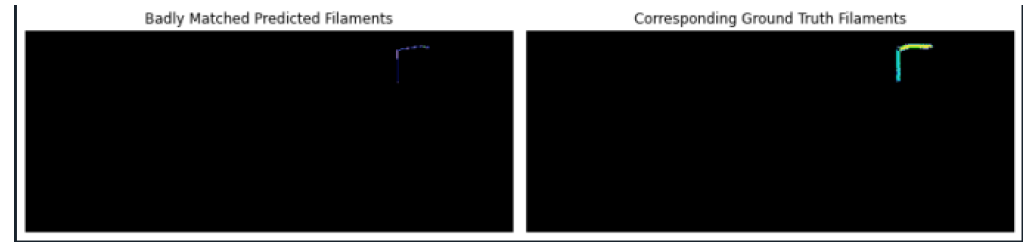


(c) Plot highlighting the falsely matched pixels.

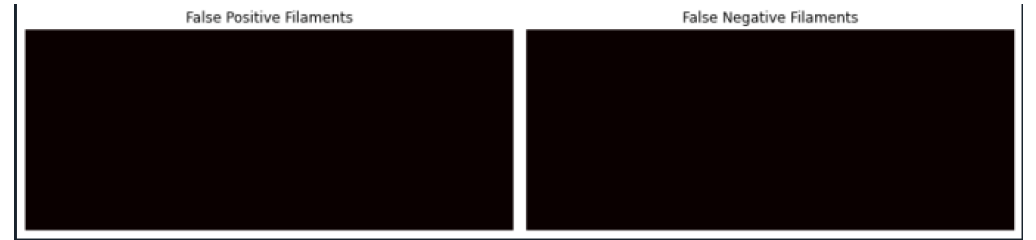
Fig 6. Performance of GraFT using the angle and thickness constraints.



(a) Predicted image from in-silico data 1 extended with closed shapes, using the angle and thickness constraints.



(b) Plot highlighting the badly matched filaments.



(c) Plot highlighting the falsely matched pixels.

Fig 7. Performance of GraFT using the angle, intensity and thickness constraints.

Table 1 presents the quantitative assessment of the performance. The true positives (TPs), false positives (FPs), and false negatives (FNs) for each filament match are counted and added together to generate the overall metrics. Based on these numbers, the cDFS algorithm with the angle and thickness constraints outperforms the cDFS with all three constraints. This is further demonstrated by the F1 Score and Intersection over Union (IoU). Nevertheless, both cDFS algorithm have a F1 Score and IoU score greater than 0.9, indicating a strong performance overall.

Table 1. Overall Metrics: Comparison of Constraints

Metric	Angle and Thickness Constraints	Angle, Intensity, and Thickness Constraints
Total True Positive	2735	2707
Total False Positive	62	90
Total False Negative	62	90
Overall Precision	0.9778333929209868	0.9678226671433678
Overall Recall	0.9778333929209868	0.9678226671433678
Overall F1 Score	0.9778333929209868	0.9678226671433678
Overall IoU	0.956628191675411	0.9376515413924489

Metrics calculated based on the angle and thickness constraints in comparison with the angle, intensity, and thickness constraints. Values include precision, recall, F1 score, and Intersection over Union (IoU).

Table 2 provides a qualitative assessment of the performance. The true positives (TPs), true negatives (TNs), false positives (FPs), and false negatives (FNs) are calculated based on the matches between the labels. Using the TNs, the Matthews correlation coefficient (MCC) is also computed. When comparing the two cDFS algorithms, the one with all the constraints performs better than the one with only the angle and thickness constraints. In fact, the former achieves a perfect score, with no false positives or false negatives, as it correctly matched all the predicted filaments to the ground truth filaments. Nevertheless, the cDFS with angle and thickness constraints still performs well, with both the F1 Score and MCC exceeding 0.95.

Table 2. Metrics: Comparison of Constraints

Metric	Angle and Thickness Constraints	Angle, Intensity, and Thickness Constraints
True Positive	21	21
False Positive	1	0
False Negative	0	0
True Negative	440	420
Precision	0.9545454545454546	1.0
Recall	1.0	1.0
F1 Score	0.9767441860465117	1.0
MCC	0.9759000729485331	1.0

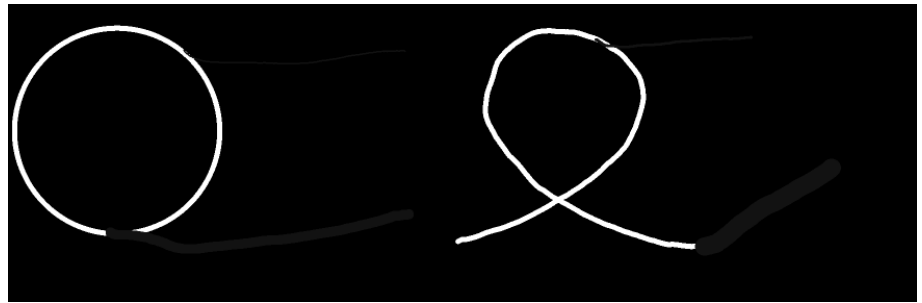
Metrics comparing angle and thickness constraints against angle, intensity, and thickness constraints, including precision, recall, F1 score, and Matthews Correlation Coefficient (MCC).

3.2 In silico data 2

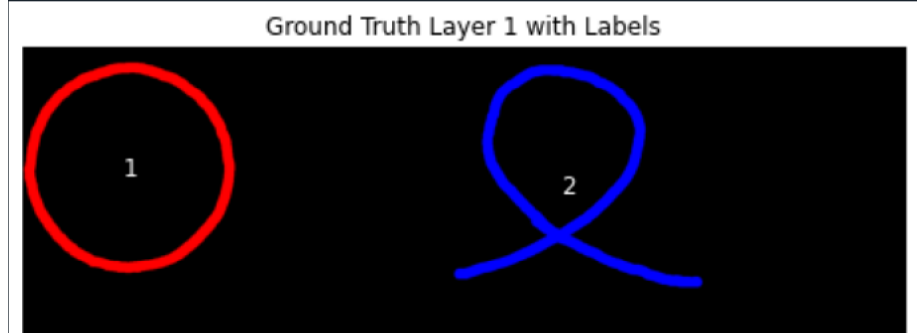
The second synthetic image was created using an Android tablet, this time with the *ibisPaintX* application and a digital pen (Fig 8). Unlike the previous tool, this pen did not introduce any artifacts or noise, resulting in filaments with consistent intensity throughout the image. Additionally, overlapping structures were incorporated into the shapes to further test the framework’s ability to handle more complex geometries.

To evaluate the robustness of the parameters across different shapes, we maintained the same settings used in the *In Silico Data 1* section. Specifically, the intensity tolerance remained at 0.17, the thickness tolerance at 0.27, and the `eps` parameter at 200. The coefficients were also unchanged, with the angle coefficient set at 5, thickness at 3, intensity at 3, and the score threshold at 8.

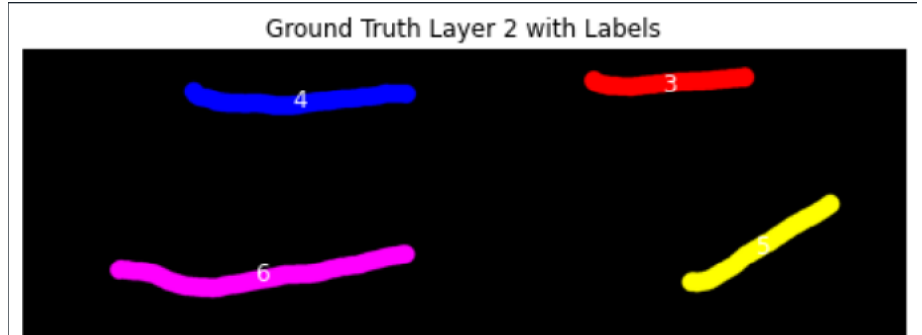
As the second synthetic dataset contained no noise, we were able to skip the pre-processing steps that were previously required. This allowed for direct testing of the tracing algorithm and parameter viability in a noise-free environment.



(a) Manually drawn test data.



(b) Ground truth image layer 1.



(c) Ground truth image layer 2.

Fig 8. Synthetic image 2.

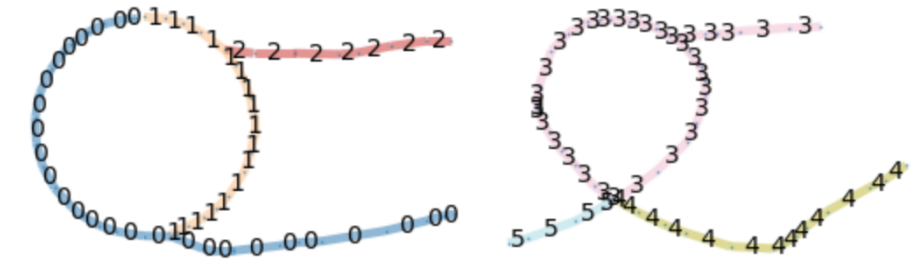
3.2.1 Performance

Original GraFT: The original GraFT framework, without any modifications, exhibited a mix of over-segmentation and under-segmentation in its labeling of filaments (Fig. 9a).

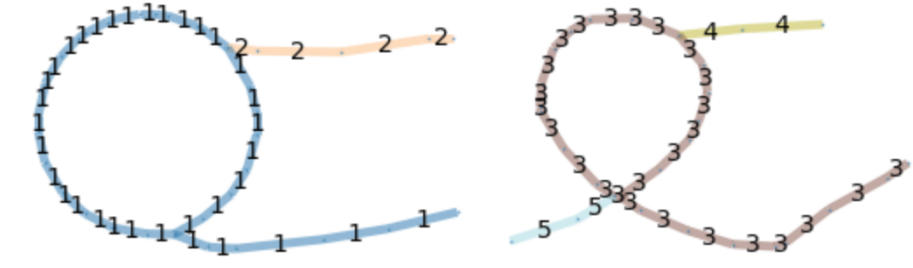
In the left closed shape, the algorithm incorrectly assigned two separate labels to what should have been a single structure. Additionally, the bottom filament was incorrectly grouped with the same structure, further contributing to mislabeling.

The shape on the right also showed a combination of under-segmentation and over-segmentation, where certain filaments were either split into too many segments or grouped incorrectly.

However, once we introduced the extension for handling closed shapes, the issue of over-segmentation was significantly improved, leading to more accurate filament tracing (Fig. 9b).



(a) Original GraFT performance.



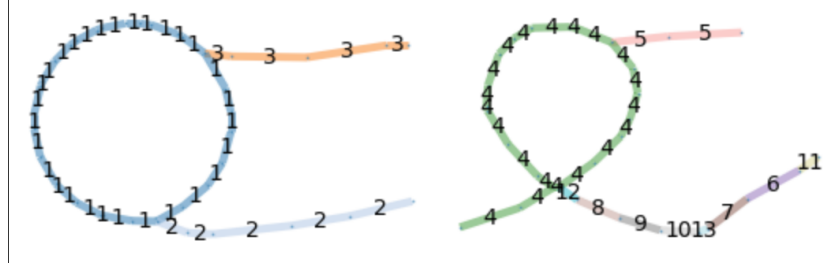
(b) GraFT performance extended with closed shapes.

Fig 9. Performance of GraFT.

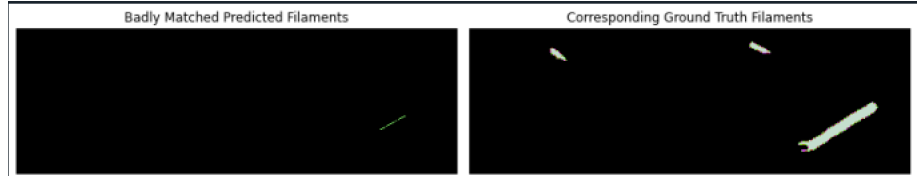
Extended GraFT: The extended GraFT framework, utilizing all the constraints—angle, intensity, and thickness—successfully labeled the left structure correctly, addressing the segmentation issues observed in the original version (Fig. 10a).

However, the right structure still experienced over-segmentation, particularly in the bottom-right filament, where the algorithm assigned more labels than necessary.

In addition, Fig. 10b highlights areas where filaments were poorly matched. The accuracy of the algorithm in these regions is not fully developed. For example, overlaps between filaments 1 and 3, as well as between filaments 4 and 5, were incorrectly detected, indicating room for improvement in how the framework handles such overlaps.



(a) Predicted image from in-silico data 2 , fully extended.



(b) Plot highlighting the badly matched filaments.



(c) Plot highlighting the falsely matched pixels.

Fig 10. Performance GraFT using the angle, intensity and thickness constraint..

Table 3. Overall Metrics: Comparison of Constraints

Metric	Angle Constraint	Angle, Intensity, and Thickness Constraint
Total True Positive	2614	3174
Total False Positive	1076	516
Total False Negative	1076	516
Overall Precision	0.7084010840108401	0.8601626016260162
Overall Recall	0.7084010840108401	0.8601626016260162
Overall F1 Score	0.7084010840108401	0.8601626016260162
Overall IoU	0.5484683172471674	0.7546362339514978

Metrics comparing the angle constraint with the angle, intensity, and thickness constraint, including precision, recall, F1 score, and Intersection over Union (IoU).

Table 4. Metrics Comparison: Angle Constraint vs. Angle, Intensity, and Thickness Constraint

Metric	Angle Constraint	Angle, Intensity, and Thickness Constraint
True Positive	4	6
False Positive	1	7
False Negative	2	0
True Negative	23	65
Precision	0.8	0.46153846153846156
Recall	0.6666666666666666	1.0
F1 Score	0.7272727272727272	0.631578947368421
MCC	0.6708203932499369	0.6454972243679028

Metrics comparing the angle constraint with the angle, intensity, and thickness constraint, including precision, recall, F1 score, and Matthews Correlation Coefficient (MCC).

Angle constraints vs all three constraints: Table 3 presents the quantitative assessment of performance, where the true positives (TPs), false positives (FPs), and false negatives (FNs) for each filament match are counted and combined to produce the overall metrics.

Based on these numbers, the cDFS algorithm that incorporates all the constraints—angle, intensity, and thickness—significantly outperforms the version using only the angle constraint in terms of overall F1 Score and Intersection over Union (IoU). This indicates that, when all constraints are considered, the algorithm is better able to trace filaments accurately.

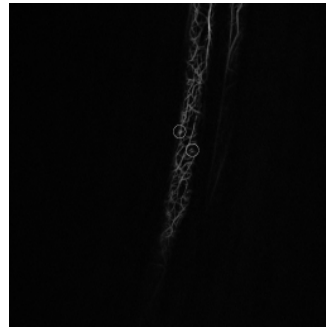
However, Table 4 provides a different perspective when evaluating the performance based solely on the matching between filaments. In this case, the cDFS algorithm using only the angle constraint shows fewer false positives, suggesting that it was less prone to over-segmentation than the algorithm with all the constraints. The F1 Score and Matthews correlation coefficient (MCC) further support this, showing that the cDFS with only the angle constraint outperformed the version with all constraints in terms of filament matching accuracy.

3.3 Biological data

For the biological data analysis, we used a seedling image from the bottom region of the plant. The pre-processing parameters, as defined by Østerlund, were kept unchanged to maintain consistency with the original framework.

When applying both the original and extended GraFT frameworks to the biological image, we observed that not all filaments were successfully segmented. This outcome appeared to be dependent on the parameter values used during the pre-processing step. The segmentation accuracy was sensitive to these settings, influencing the framework's ability to accurately trace all filaments in the image.

Unfortunately, due to time constraints, we were unable to fully adjust and optimize the pre-processing parameters in conjunction with the parameters introduced in the extended GraFT framework. As a result, we were unable to quantify the performance of the algorithm on this biological dataset.



(a) Biological data:
Arabidopsis thaliana seedling
bottom with two circular
filament shapes.



(b) Original GraFT
performance.



(c) Extended GraFT
performance.

Fig 11. GraFT on biological data.

4 Discussion

The original GraFT framework performed well when tracing simple filament structures, as demonstrated with shapes h and i in Fig. 2a. These simple shapes, lacking closed structures or significant overlap, were correctly traced without issue. However, as expected, the original GraFT struggled when faced with more complex geometries, particularly closed shapes and overlapping structures (Fig. 2). In these cases, the framework was unable to accurately assign the correct filaments, highlighting a key limitation in its design.

One of the major improvements introduced by our extension to the GraFT framework was its ability to handle closed shapes and overlapping filaments. This improvement was made possible by extending the framework's tracing algorithm and incorporating a more nuanced scoring system. The new system considers multiple criteria—angle, intensity, and thickness—to make better decisions when assigning filaments. However, while these additions improve performance, they also come with challenges. The accuracy of the scoring system is heavily dependent on the threshold values for each parameter, and tuning these values can be both time-consuming and difficult. The introduction of more parameters has, therefore, increased the complexity of optimizing the framework.

Despite these advancements, further work is needed to fully validate the extension's performance, particularly on real biological data. Due to time constraints, tracking filaments over time was not implemented in this project, and additional validation sets must be created to rigorously test the framework on more diverse datasets. Testing the framework on biological data is crucial to ensure its robustness in real-world applications, where the complexity of filamentous structures may vary.

An important next step in improving the usability of the extended GraFT framework would be the development of a graphical user interface (GUI), similar to CytoSeg 2.0. A GUI would allow users to dynamically adjust parameter values and immediately visualize their effects on the tracing results. This could greatly simplify the process of tuning parameters and make the tool more accessible to researchers who may not have extensive experience in programming or algorithm optimization.

In summary, while our extension significantly improves the original GraFT framework's ability to handle closed shapes and complex overlaps, it introduces more parameters that require careful tuning. The addition of a GUI could streamline this process, making the tool more user-friendly and adaptable to different types of biological datasets. Further validation on real biological data is necessary to fully assess the extension's effectiveness.

References

1. Fletcher DA, Mullins RD. Cell mechanics and the cytoskeleton. *Nature*. 2010;463(7280):485–492. doi:10.1038/nature08908.
2. Schaedel L, Lorenz C, Schepers AV, Klumpp S, Köster S. Vimentin intermediate filaments stabilize dynamic microtubules by direct interactions. *Nature Communications*. 2021;12(1):3799. doi:10.1038/s41467-021-23523-z.
3. Mehrbod M, Mofrad MRK. On the Significance of Microtubule Flexural Behavior in Cytoskeletal Mechanics. *PLOS ONE*. 2011;6(10):e25627. doi:10.1371/journal.pone.0025627.
4. Gittes F, Mickey B, Nettleton J, Howard J. Flexural rigidity of microtubules and actin filaments measured from thermal fluctuations in shape. *The Journal of Cell Biology*. 1993;120(4):923–934. doi:10.1083/jcb.120.4.923.
5. Infante E, Etienne-Manneville S. Intermediate filaments: Integration of cell mechanical properties during migration. *Frontiers in Cell and Developmental Biology*. 2022;10. doi:10.3389/fcell.2022.951816.
6. Breuer D, Ivakov A, Sampathkumar A, Hollandt F, Persson S, Nikoloski Z. Quantitative analyses of the plant cytoskeleton reveal underlying organizational principles. *Journal of The Royal Society Interface*. 2014;11(97):20140362. doi:10.1098/rsif.2014.0362.
7. Cooper GM. Structure and Organization of Actin Filaments. In: *The Cell: A Molecular Approach*. 2nd edition. Sinauer Associates; 2000. Available from: <https://www.ncbi.nlm.nih.gov/books/NBK9908/>.
8. Dominguez R, Holmes KC. Actin structure and function. *Annual Review of Biophysics*. 2011;40:169–186. doi:10.1146/annurev-biophys-042910-155359.
9. Xu L, Cao L, Li J, Staiger CJ. Cooperative actin filament nucleation by the Arp2/3 complex and formins maintains the homeostatic cortical array in *Arabidopsis* epidermal cells. *The Plant Cell*. 2024;36(3):764–789. doi:10.1093/plcell/koad301.
10. Cao L, Henty-Ridilla JL, Blanchoin L, Staiger CJ. Profilin-Dependent Nucleation and Assembly of Actin Filaments Controls Cell Elongation in *Arabidopsis*1[OPEN]. *Plant Physiology*. 2016;170(1):220–233. doi:10.1104/pp.15.01321.
11. Dominguez R. Actin filament nucleation and elongation factors–structure–function relationships. *Critical Reviews in Biochemistry and Molecular Biology*. 2009;44(6):351–366. doi:10.3109/10409230903277340.
12. Gao W, Wu H, Siddiqui MK, Baig AQ. Study of biological networks using graph theory. *Saudi Journal of Biological Sciences*. 2018;25(6):1212–1219. doi:10.1016/j.sjbs.2017.11.022.
13. Chasman D, Fotuhi Siahpirani A, Roy S. Network-based approaches for analysis of complex biological systems. *Current Opinion in Biotechnology*. 2016;39:157–166. doi:10.1016/j.copbio.2016.04.007.

14. Breuer D, Nowak J, Ivakov A, Somssich M, Persson S, Nikoloski Z. System-wide organization of actin cytoskeleton determines organelle transport in hypocotyl plant cells. *Proceedings of the National Academy of Sciences*. 2017;114(28):E5741–E5749. doi:10.1073/pnas.1706711114.
15. Wang L, Qiu T, Yue J, Guo N, He Y, Han X, et al. Arabidopsis ADF1 is Regulated by MYB73 and is Involved in Response to Salt Stress Affecting Actin Filament Organization. *Plant & Cell Physiology*. 2021;62(9):1387–1395. doi:10.1093/pcp/pcab081.
16. Gurari D, Theriault D, Sameki M, Isenberg B, Pham T, Purwada A, et al. How to Collect Segmentations for Biomedical Images? A Benchmark Evaluating the Performance of Experts, Crowdsourced Non-experts, and Algorithms. *Proceedings - 2015 IEEE Winter Conference on Applications of Computer Vision, WACV 2015*. 2015; p. 1169–1176. doi:10.1109/WACV.2015.160.
17. Danuser G. Computer vision in cell biology. *Cell*. 2011;147(5):973–978. doi:10.1016/j.cell.2011.11.001.
18. Alioscha-Perez M, Benadiba C, Goossens K, Kasas S, Dietler G, Willaert R, et al. A Robust Actin Filaments Image Analysis Framework. *PLOS Computational Biology*. 2016;12(8):e1005063. doi:10.1371/journal.pcbi.1005063.
19. Nikoloski Z, Østerlund I, Ma Z, Miao Y, Persson S. A Robust Network-based Spatiotemporal Analysis of Filamentous Structures; 2024.
20. Xu T, Vavylonis D, Tsai FC, Koenderink GH, Nie W, Yusuf E, et al. SOAX: A software for quantification of 3D biopolymer networks. *Scientific Reports*. 2015;5(1):9081. doi:10.1038/srep09081.
21. Xu T, Langouras C, Koudehi MA, Vos BE, Wang N, Koenderink GH, et al. Automated Tracking of Biopolymer Growth and Network Deformation with TSOAX. *Scientific Reports*. 2019;9(1):1717. doi:10.1038/s41598-018-37182-6.
22. Xu T, Li H, Shen T, Ojkic N, Vavylonis D, Huang X. EXTRACTION AND ANALYSIS OF ACTIN NETWORKS BASED ON OPEN ACTIVE CONTOUR MODELS. *Proceedings / IEEE International Symposium on Biomedical Imaging: from nano to macro IEEE International Symposium on Biomedical Imaging*. 2011;2011:1334–1340. doi:10.1109/ISBI.2011.5872647.
23. Hauke L, Primeßnig A, Eltzner B, Radwitz J, Huckemann SF, Rehfeldt F. FilamentSensor 2.0: An open-source modular toolbox for 2D/3D cytoskeletal filament tracking. *PLOS ONE*. 2023;18(2):e0279336. doi:10.1371/journal.pone.0279336.
24. Flormann DAD, Schu M, Terriac E, Thalla D, Kainka L, Koch M, et al. A novel universal algorithm for filament network tracing and cytoskeleton analysis. *The FASEB Journal*. 2021;35(5):e21582. doi:10.1096/fj.202100048R.
25. Dirnberger M, Kehl T, Neumann A. NEFI: Network Extraction From Images. *Scientific Reports*. 2015;5(1):15669. doi:10.1038/srep15669.
26. Nowak J, Gennermann K, Persson S, Nikoloski Z. CytoSeg 2.0: automated extraction of actin filaments. *Bioinformatics*. 2020;36(9):2950–2951. doi:10.1093/bioinformatics/btaa035.

27. Breuer D, Nikoloski Z. DeFiNe: an optimisation-based method for robust disentangling of filamentous networks. *Scientific Reports*. 2015;5(1):18267. doi:10.1038/srep18267.
28. Schindelin J, Arganda-Carreras I, Frise E, Kaynig V, Longair M, Pietzsch T, et al. Fiji: an open-source platform for biological-image analysis. *Nature Methods*. 2012;9(7):676–682. doi:10.1038/nmeth.2019.
29. Mary H, Rueden C. hadim/FilamentDetector: FilamentDetector-0.4.7; 2019. Available from: <https://zenodo.org/records/2544848>.
30. Kapoor V, Hirst WG, Hentschel C, Preibisch S, Reber S. MTrack: Automated Detection, Tracking, and Analysis of Dynamic Microtubules. *Scientific Reports*. 2019;9(1):3794. doi:10.1038/s41598-018-37767-1.
31. Martiel JL, Michelot A, Boujemaa-Paterski R, Blanchoin L, Berro J. Force Production by a Bundle of Growing Actin Filaments Is Limited by Its Mechanical Properties. *Biophysical Journal*. 2020;118(1):182–192. doi:10.1016/j.bpj.2019.10.039.
32. Østerlund I, Persson S, Nikoloski Z. Tracing and tracking filamentous structures across scales: A systematic review. *Computational and Structural Biotechnology Journal*. 2023;21:452–462. doi:10.1016/j.csbj.2022.12.023.
33. Tarjan R. Depth-first search and linear graph algorithms. In: 12th Annual Symposium on Switching and Automata Theory (swat 1971); 1971. p. 114–121. Available from: <https://ieeexplore.ieee.org/document/4569669>.
34. Østerlund I. Arabidopsis Thaliana seedling cells; maturing and drug treatment; 2024. Available from: <https://zenodo.org/records/10476058>.
35. Van Rossum G, Drake FL. Python 3 Reference Manual. Scotts Valley, CA: CreateSpace; 2009.

I am aware of and understand the university's policy on plagiarism, and I certify that this term paper is my own work, except where indicated by referencing. Contributions from colleagues (e.g., prior work, obtaining samples or data, etc) are explicitly referenced in the text. The work presented in this paper has not been submitted in support of another course, degree, or qualification from this or any other university or institute of learning.

A handwritten signature in blue ink, appearing to be 'S. K. Singh', written in a cursive style.